

Code-Layout, Readability and Reusability

Date	02 October 2026
Team ID	SWTID-2026-2606
Project Name	PocketSmart AI: Your Smart Budget & Recommendation Assistant
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	4-space indentation is used consistently.
2	Proper File Structure	Files and folders are logically organized	Partial	templates/, static/ and .env are separated, but routes, models, auth and AI logic are all in app.py.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Names like budget_input, search_terms and API_KEY show their purpose.
4	Function / Method Names	Functions are descriptively named	Yes	Names like get_home_recommendations clearly show what each function does.
5	Code Comments	Inline and block comments explain logic	Partial	Section comments and docstrings exist, but not for every function.
6	Modular Design	Code is split into reusable functions/modules	Partial	One function per planner plus Pydantic models and templates; not yet split into separate files.
7	No Redundant Code	Duplicate or unused code is removed	Partial	Shopping-link code is repeated in each planner and some imports are duplicated.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	API-key check and try/except with HTTP 500 errors; the broad Exception catch could be narrowed.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	Authentication module (register, login, logout, /token)	Python, FastAPI, passlib (bcrypt), JWT	Guards all planner, dashboard and history pages through the current-user dependency	High
2	Pydantic input models (HomeBudgetInput, PartyBudgetInput, JewelryBudgetInput, RegisterUser)	Python, Pydantic	Request validation in the planner and registration routes and the recommendation functions	High
3	Gemini prompt and response handling (prompt, model call, JSON extraction)	Python, Google Generative AI (Gemini)	Home, Party and Jewelry recommendation functions	High
4	Shopping-link builder (search URLs from AI search terms)	Python, urllib.parse	Home, Party and Jewelry planners; currently repeated in each, so it can be made one shared function	Medium
5	Page templates and shared stylesheet (templates/, static/styles.css)	HTML, CSS, Jinja2	Dashboard, history, planner, login and register pages	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	3 / 5	Templates, static files and secrets are separated, but the backend lives in a single app.py.
Readability	4 / 5	Consistent indentation, clear names and docstrings make the code easy to follow.
Reusability	3 / 5	Auth, models, Gemini handling and templates are reusable; shopping-link code is repeated per planner.
Documentation / Comments	3 / 5	Section comments and docstrings exist; more comments are needed inside the larger functions.
Overall Score	3.3 / 5	Working, readable code; next steps are to split app.py into modules and share the repeated helpers.